



# 矩阵计算与优化

## (最优化--有约束优化)

授课人：陈硕



南京大学  
NANJING UNIVERSITY



## 大纲

- 约束极值及最优性条件
- 约束极值问题的算法





## 矩阵计算与优化 (最优化--有约束优化)

1、约束极值问题的表示  $\min f(x)$

$$s.t. \quad \begin{cases} h_i(x) = 0 & i = 1, 2, \dots, m \\ g_j(x) \leq 0 & j = 1, 2, \dots, l \end{cases}$$

记  $h(x) = (h_1(x), h_2(x), \dots, h_m(x))^T$ ,

$g(x) = (g_1(x), g_2(x), \dots, g_l(x))^T$ ,

则约束极值问题可记为  $\min f(x)$

$$s.t. \quad \begin{cases} h(x) = 0 \\ g(x) \leq 0 \end{cases}$$

令  $Q = \{x | h(x) = 0, g(x) \leq 0\}$ , 称  $Q$  为此约束极值问题的可行域。





## 矩阵计算与优化 (最优化--有约束优化)

$$\min f(x)$$

$$\text{s.t.} \quad \begin{cases} h_i(x) = 0 & i = 1, 2, \dots, m \\ g_j(x) \leq 0 & j = 1, 2, \dots, l \end{cases}$$

$$\because h_i(x) = 0 \Leftrightarrow \begin{cases} h_i(x) \leq 0 \\ -h_i(x) \leq 0 \end{cases}$$

$\therefore$  约束极值问题也可记为

$$\min f(x)$$

$$\text{s.t.} \quad g(x) \leq 0$$





### 2 约束极值及最优性条件——Kuhn-Tucker 条件

#### (1) 等式约束性问题的最优性条件

考虑

$$\min f(x)$$

$$\text{s.t. } h(x)=0$$

回顾高等数学中所学的条件极值：

**问题** 求  $z = f(x,y)$  极值，在  $\phi(x,y)=0$  的条件下。

即：  $\min f(x,y)$

$$\text{s.t. } \phi(x,y)=0$$

引入Lagrange乘子：  $\lambda$

Lagrange函数  $L(x,y;\lambda)=f(x,y)+\lambda \phi(x,y)$





## 矩阵计算与优化 (最优化--有约束优化)

若 $(x^*, y^*)$ 是条件极值, 则存在 $\lambda^*$ , 使得

$$\begin{cases} f_x(x^*, y^*) + \lambda^* \phi_x(x^*, y^*) = 0 \\ f_y(x^*, y^*) + \lambda^* \phi_y(x^*, y^*) = 0 \\ \phi(x^*, y^*) = 0 \end{cases}$$

推广到多元情况, 可得到对于等式约束的情况:

分量形式: 
$$\begin{cases} \min f(x) \\ \text{s.t. } h_j(x) = 0, \quad j = 1, 2, \dots, l \end{cases}$$

若 $x^*$ 是其的最优解, 则存在 $v^* \in R^l$  使

$$\nabla f(x^*) + \sum_{j=1}^l v_j^* \nabla h_j(x^*) = 0$$

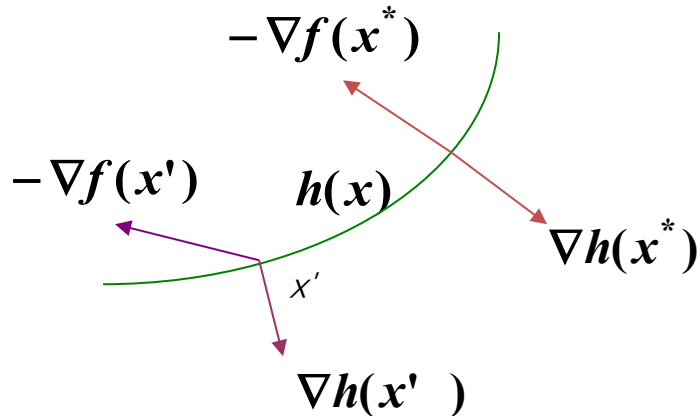




## 矩阵计算与优化 (最优化--有约束优化)

$$\nabla f(x^*) + \sum_{j=1}^l \nu_j^* \nabla h_j(x^*) = 0$$

**几何意义：**考虑一个约束的情况：



这里 $x^*$ 最优,  $\nabla f(x^*)$ 与  
 $\nabla h(x^*)$ 共线, 而 $x'$ 非最优  
 $\nabla f(x')$ 与 $\nabla h(x')$ 不共线,

最优性条件即：

$$\nabla f(x^*) = - \sum_{j=1}^l \nu_j^* \nabla h_j(x^*)$$





# 矩阵计算与优化 (最优化--有约束优化)

## (2) 不等式约束极值问题的最优性条件

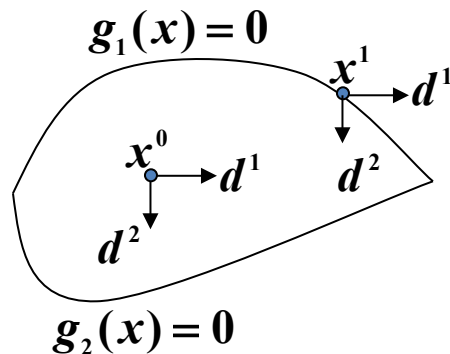
$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & g(x) \leq 0 \end{aligned} \quad (1)$$

可行域为  $Q = \{x | g(x) \leq 0\}$ 。

①可行方向与积极约束:

可行方向:

设  $x^0 \in Q$ ,  $d$  为一个向量。如果存在实数  $\bar{\lambda} > 0$ , 使得对任意的  $\lambda \in [0, \bar{\lambda}]$  有  $x^0 + \lambda d \in Q$ , 则称  $d$  为  $x^0$  处的一个可行方向。







## 矩阵计算与优化 (最优化--有约束优化)

**积极约束:** 设点  $\bar{x} \in Q$ , 对于不等式约束  $g_i(x) \leq 0$ , 如果  $g_i(\bar{x}) = 0$ , 则称  $g_i(x) \leq 0$  是点  $\bar{x}$  处的积极约束。

或 **起作用约束 (紧约束\积极约束\有效约束)**。

记  $I(\bar{x}) = \{i | g_i(\bar{x}) = 0, 1 \leq i \leq l\}$ , 称  $I(\bar{x})$  为点  $\bar{x}$  处的积极约束指标集。

**例:** 设  $g_1(x) = \sqrt{2}x_1^2 - x_2 \leq 0, g_2(x) = x_1^2 + x_2^2 - 1 \leq 0,$   
 $g_3(x) = -x_1 \leq 0$ 。令  $\bar{x} = (\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2})^T$ , 求点  $\bar{x}$  的积极约束指标集。

**解:**  $\because g_1(\bar{x}) = \sqrt{2} \times (\frac{\sqrt{2}}{2})^2 - \frac{\sqrt{2}}{2} = 0,$

$$g_2(\bar{x}) = (\frac{\sqrt{2}}{2})^2 + (\frac{\sqrt{2}}{2})^2 - 1 = 0,$$

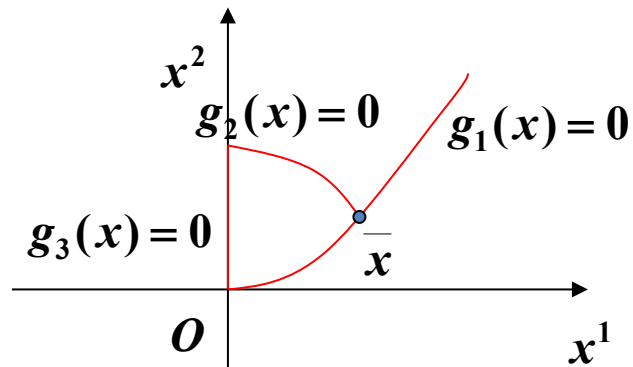




## 矩阵计算与优化 (最优化--有约束优化)

$$g_3(\bar{x}) = -\frac{\sqrt{2}}{2} < 0。$$

$$\therefore I(\bar{x}) = \{1, 2\}。$$



②如何判断一个方向是可行方向?





## 矩阵计算与优化 (最优化--有约束优化)

**定理1:** 给定点  $\bar{x} \in Q$ , 记点  $\bar{x}$  的积极约束指标集为  $I(\bar{x})$ 。给定向量  $d$ , 如果对任意的  $i \in I(\bar{x})$  有  $\nabla g_i(\bar{x})^T d < 0$ , 则  $d$  是点  $\bar{x}$  的可行方向。

**证明:** 令  $x' = \bar{x} + td, t > 0$ 。则对任意的  $i \in I(\bar{x})$ , 有

$$\begin{aligned} g_i(x') &= g_i(\bar{x}) + t \nabla g_i(\bar{x})^T d + o(\|td\|^2) \\ &= t \nabla g_i(\bar{x})^T d + o(\|td\|^2) < 0 \end{aligned}$$

$\therefore x' \in Q$ , 即  $d$  为可行方向。

**可行下降方向:**

设点  $\bar{x} \in Q$ , 给定向量  $d$ , 如果  $d$  既是点  $\bar{x}$  处的可行方向, 又是该点的下降方向, 则称  $d$  为点  $\bar{x}$  处的可行下降方向。





## 矩阵计算与优化 (最优化--有约束优化)

定理2: 给定点  $\bar{x} \in Q$ , 记点  $\bar{x}$  的积极约束指标集为  $I(\bar{x})$ 。给定向量  $d$ , 如果  $d$  满足

$$\begin{cases} \nabla g_i(\bar{x})^T d < 0 & i \in I(\bar{x}) \\ \nabla f(\bar{x})^T d < 0 \end{cases}$$

则向量  $d$  是点  $\bar{x}$  处的可行下降方向。 证略

③极值点的必要条件:

定理3: 设  $x^* \in Q$ ,  $I(x^*)$  是其积极约束指标集。

$f(x)$  和  $g_i(x) (i \in I(x^*))$  在点  $x^*$  处可微,

$g_i(x) (i \notin I(x^*))$  在点  $x^*$  处连续。

如果  $x^*$  是约束极值问题(1)的局部极小点, 则在点  $x^*$  处没有可行下降方向。





### 3. K-T条件 (库恩-塔克条件)

设 $I(x^*)$ 是其积极约束指标集, 点 $x^*$ 是约束极值问题(1)的局部极小点, 则在点 $x^*$ 处不存在下降方向 $d$ 。

则由定理2可知, 不存在向量 $d$ , 使下式成立

$$\begin{cases} \nabla g_i(x^*)^T d < 0 & i \in I(x^*) \\ \nabla f(x^*)^T d < 0 \end{cases}。$$

分析:

(1) 如果 $I(x^*)$ 中只有一个指标, 不妨设 $g_1(x)$ 为积极约束。

则不存在向量 $d$ 使得

$$\begin{cases} \nabla g_1(x^*)^T d < 0 \\ \nabla f(x^*)^T d < 0 \end{cases}$$

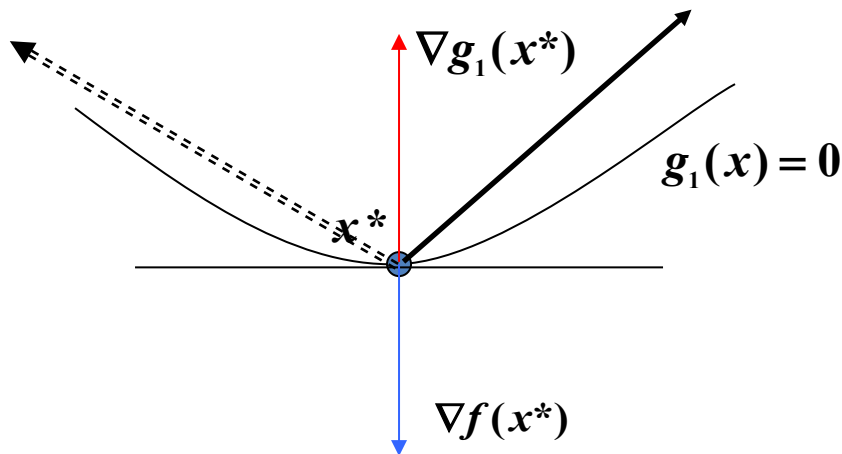
成立。





## 矩阵计算与优化 (最优化--有约束优化)

则不存在向量  $d$  使得  
$$\begin{cases} \nabla g_1(x^*)^T d < 0 \\ \nabla f(x^*)^T d < 0 \end{cases}$$
 成立。



则有  $\nabla f(x^*) = -\lambda \nabla g_1(x^*), \lambda > 0$ 。

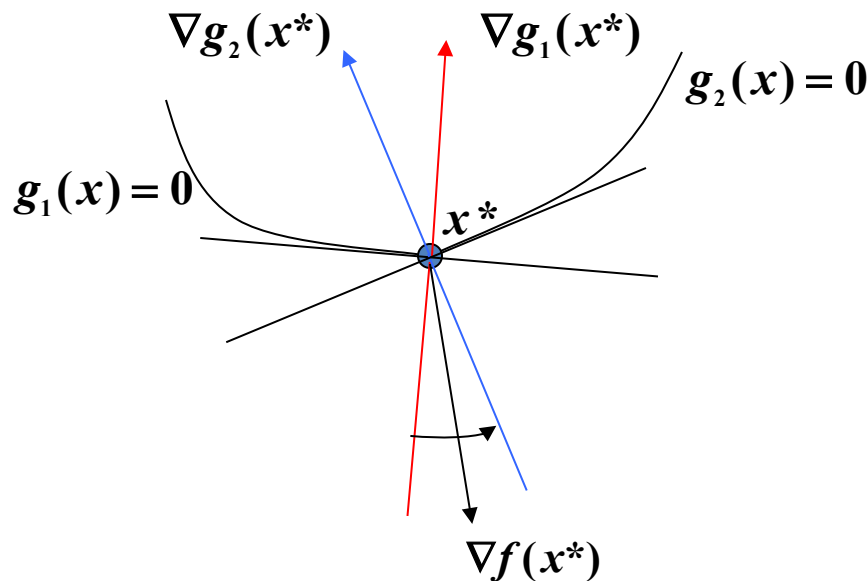
即  $\nabla f(x^*) + \lambda \nabla g_1(x^*) = 0$ 。





## 矩阵计算与优化 (最优化--有约束优化)

- (2) 如果  $I(x^*)$  中有两个指标, 不妨设  $g_1(x)$  和  $g_2(x)$  为积极约束。  
并设  $\nabla g_1(x^*)$  和  $\nabla g_2(x^*)$  线性无关。



则 存在  $\lambda_1, \lambda_2 > 0$ , 使得  $\nabla f(x^*) = -\lambda_1 \nabla g_1(x^*) - \lambda_2 \nabla g_2(x^*)$ 。





## 矩阵计算与优化 (最优化--有约束优化)

$$\therefore \nabla f(x^*) + \lambda_1 \nabla g_1(x^*) + \lambda_2 \nabla g_2(x^*) = 0。$$

(3) 一般情况：设 $\{\nabla g_i(x^*) | i \in I(x^*)\}$ 线性无关。

则存在非负实数  $\lambda_i (i \in I(x^*))$ , 使得

$$\nabla f(x^*) + \sum_{i \in I(x^*)} \lambda_i \nabla g_i(x^*) = 0 \quad (2)$$

(2)式可改写为

$$\left\{ \begin{array}{l} \nabla f(x^*) + \sum_{i=1}^l \lambda_i \nabla g_i(x^*) = 0 \\ \lambda_i g_i(x^*) = 0, \lambda_i \geq 0, i = 1, 2, \dots, l \end{array} \right. \quad (3)$$







## 矩阵计算与优化 (最优化--有约束优化)

$$\begin{cases} \lambda_i g_i(x^*) = 0, i = 1, 2, \dots, l \\ \lambda_i \geq 0, i = 1, 2, \dots, l \end{cases} \Rightarrow \begin{cases} \lambda_i = 0, & g_i(x^*) < 0; \\ \lambda_i \geq 0, & g_i(x^*) = 0; \end{cases}$$

**定理4(K-T条件):** 设  $x^* \in Q$ ,  $f(x)$  和  $g_i(x) (i \in I(x^*))$  在  $x^*$  处可微,  
 $g_i(x) (i \notin I(x^*))$  在点  $x^*$  处连续,  $\{\nabla g_i(x^*) | i \in I(x^*)\}$   
线性无关。若  $x^*$  是约束极值问题 (1) 的局部极小点,  
则存在一组实数  $\lambda_i$  使其满足

$$\begin{cases} \nabla f(x^*) + \sum_{i=1}^l \lambda_i \nabla g_i(x^*) = 0 \\ \lambda_i g_i(x^*) = 0, \lambda_i \geq 0, \quad i = 1, 2, \dots, l \end{cases} \quad (*)$$

(\*) 式称为 K-T 条件 (库恩-塔克条件), 满足 (\*) 式的点  
称为 K-T 点。





(4) 对于有等式约束的极值 问题

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & \begin{cases} h(x) = 0 \\ g(x) \leq 0 \end{cases} \end{aligned}$$

K-T条件可写为

$$\begin{cases} \nabla f(x^*) + \sum_{j=1}^m u_j \nabla h_j(x^*) + \sum_{i=1}^l \lambda_i \nabla g_i(x^*) = 0 \\ \lambda_i g_i(x^*) = 0, \lambda_i \geq 0, \quad i = 1, 2, \dots, l \end{cases} \quad (**)$$





### K-T点的计算

例：求约束极值问题

$$\begin{aligned} \min \quad & f(x) = x_1^2 + x_2^2 - 6x_1 - 6x_2 + 8 \\ \text{s.t.} \quad & \begin{cases} x_1 + x_2 \leq 4 \\ x_1 \geq 0 \\ x_2 \geq 0 \end{cases} \end{aligned}$$

的K-T点。

解：  $\nabla f(x) = 2[x_1 - 3, x_2 - 3]^T$ 。

$$g_1(x) = x_1 + x_2 - 4, \quad \nabla g_1(x) = [1, 1]^T$$

$$g_2(x) = -x_1, \quad \nabla g_2(x) = [-1, 0]^T。$$

$$g_3(x) = -x_2, \quad \nabla g_3(x) = [0, -1]^T。$$





## 矩阵计算与优化 (最优化--有约束优化)

由 K-T 条件得

$$\begin{bmatrix} x_1 - 3 \\ x_2 - 3 \end{bmatrix} + \lambda_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \lambda_2 \begin{bmatrix} -1 \\ 0 \end{bmatrix} + \lambda_3 \begin{bmatrix} 0 \\ -1 \end{bmatrix} = 0$$

由 K-T 条件及约束条件得

$$\begin{cases} x_1 + \lambda_1 - \lambda_2 = 3 \\ x_2 + \lambda_1 - \lambda_3 = 3 \\ \lambda_1(x_1 + x_2 - 4) = 0 \\ -\lambda_2 x_1 = 0 \\ -\lambda_3 x_2 = 0 \\ x_1 + x_2 - 4 \leq 0 \\ \lambda_1, \lambda_2, \lambda_3, x_1, x_2 \geq 0 \end{cases}$$

以下分情况讨论:





## 矩阵计算与优化 (最优化--有约束优化)

(1) 若  $x_1 = x_2 = 0$ :

由  $\lambda_1(x_1 + x_2 - 4) = 0$  可得  $\lambda_1 = 0$ 。

$$\therefore \lambda_1 - \lambda_2 = 3 \Rightarrow \lambda_2 = -3$$

这与  $\lambda_2 \geq 0$  矛盾。

(2) 若  $x_1 = 0, x_2 \neq 0$ :

$$\therefore \lambda_3 = 0$$

$$\therefore \begin{cases} x_2 + \lambda_1 = 3 \\ \lambda_1 - \lambda_2 = 3 \end{cases} \Rightarrow x_2 + \lambda_2 = 0 \Rightarrow \lambda_2 = -x_2 < 0$$

这与  $\lambda_2 \geq 0$  矛盾。

(3) 若  $x_1 \neq 0, x_2 = 0$ :

$$\therefore \lambda_2 = 0$$

$$\left\{ \begin{array}{l} x_1 + \lambda_1 - \lambda_2 = 3 \\ x_2 + \lambda_1 - \lambda_3 = 3 \\ \lambda_1(x_1 + x_2 - 4) = 0 \\ -\lambda_2 x_1 = 0 \\ -\lambda_3 x_2 = 0 \\ x_1 + x_2 - 4 \leq 0 \\ \lambda_1, \lambda_2, \lambda_3, x_1, x_2 \geq 0 \end{array} \right.$$





## 矩阵计算与优化 (最优化--有约束优化)

$$\therefore \begin{cases} x_1 + \lambda_1 = 3 \\ \lambda_1 - \lambda_3 = 3 \end{cases} \Rightarrow x_1 + \lambda_3 = 0 \Rightarrow \lambda_3 = -x_1 < 0$$

这与  $\lambda_3 \geq 0$  矛盾。

(4) 若  $x_1 \neq 0, x_2 \neq 0$ :

$$\therefore \lambda_2 = \lambda_3 = 0$$

$$\therefore \begin{cases} x_1 + \lambda_1 = 3 \\ x_2 + \lambda_1 = 3 \end{cases} \Rightarrow x_1 = x_2$$

$$\text{若 } x_1 + x_2 < 4 \Rightarrow \lambda_1 = 0 \Rightarrow x_1 = x_2 = 3$$

$$\Rightarrow x_1 + x_2 = 6 > 4 \Rightarrow \text{矛盾。}$$

$$\therefore x_1 + x_2 = 4 \Rightarrow x_1 = x_2 = 2 \Rightarrow \lambda_1 = 1$$

$$\therefore [2, 2]^T \text{ 为 K-T 点。}$$

$$\left\{ \begin{array}{l} x_1 + \lambda_1 - \lambda_2 = 3 \\ x_2 + \lambda_1 - \lambda_3 = 3 \\ \lambda_1(x_1 + x_2 - 4) = 0 \\ -\lambda_2 x_1 = 0 \\ -\lambda_3 x_2 = 0 \\ x_1 + x_2 - 4 \leq 0 \\ \lambda_1, \lambda_2, \lambda_3, x_1, x_2 \geq 0 \end{array} \right.$$





### 4. 其它最优性条件

定理5(Fritz John条件):

设  $x^* \in Q$ ,  $f(x)$  和  $g_i(x) (i \in I(x^*))$  在点  $x^*$  处可微,  
 $g_i(x) (i \notin I(x^*))$  在点  $x^*$  处连续。若  $x^*$  是约束极值  
问题 (1) 的局部极小点, 则存在一组不全为零  
的非负实数  $\lambda_i$ , 使得

$$\lambda_0 \nabla f(x^*) + \sum_{i \in I(x^*)} \lambda_i \nabla g_i(x^*) = 0$$





## 矩阵计算与优化 (最优化--有约束优化)

一般的约束极值问题 (cnlp)  $\min f(x)$   
s.t.  $\begin{cases} h(x) = 0 \\ g(x) \leq 0 \end{cases}$

定理 (K-T条件): 设  $x^* \in Q$ ,  $f(x)$  和  $g_i(x) (i \in I(x^*))$  在  $x^*$  处可微,  
 $g_i(x) (i \notin I(x^*))$  在点  $x^*$  处连续,  $\{\nabla g_i(x^*) | i \in I(x^*)\}$   
线性无关。若  $x^*$  是约束极值问题 (cnlp) 的局部极小点,  
则存在一组实数  $\lambda_i$  使其满足

$$(*) \begin{cases} \nabla f(x^*) + \sum_{j=1}^m u_j \nabla h_j(x^*) + \sum_{i=1}^l \lambda_i \nabla g_i(x^*) = 0 \\ \lambda_i g_i(x^*) = 0, \lambda_i \geq 0, \quad i = 1, 2, \dots, l \end{cases}$$

(\*)式称为K-T条件 (库恩-塔克条件), 满足(\*)式的点称为K-T点。







## 大纲

- 约束极值及最优性条件
- 约束极值问题的算法





## 二、约束极值问题的算法

### （一）惩罚函数法（SUMT）

#### 1、算法思想：

将有约束优化问题转化为一系列无约束优化问题进行求解。（Sequential Unconstrained Minimization Technique - SUMT）

#### 2、算法类型：

- ☐ 外点法（外惩罚法）
- ☐ 内点法（内惩罚法）





3、问题:

$$\begin{aligned} \min f(x) \\ \text{s.t. } g_i(x) \leq 0 \quad i = 1, \dots, m \end{aligned}$$

$$\Leftrightarrow \begin{aligned} \min f(x) \\ \text{s.t. } g(x) \leq 0 \quad \text{这里 } g(x) = (g_1(x), \dots, g_m(x))^T \end{aligned}$$

$$\begin{aligned} \Leftrightarrow \min f(x) \\ \text{s.t. } x \in D \\ D = \{x \mid g(x) \leq 0\} : \text{可行点集或可行解集} \end{aligned}$$





## 矩阵计算与优化 (最优化--有约束优化)

### 4、外点法 (外部惩罚函数法)

算法思想：构造 $\varphi_k(x)$

$$\varphi_k(x) \text{ 满足: } \varphi_k(x) = f(x) \quad x \in D$$

$$\varphi_k(x) > f(x) \quad x \notin D \text{ 且 } \varphi_k(x) \uparrow (k \uparrow)$$

$$\min_{x \in D} f(x) \Rightarrow \min_{x \in R^n} \varphi_1(x) \Rightarrow \min_{x \in R^n} \varphi_2(x) \Rightarrow \cdots \Rightarrow \min_{x \in R^n} \varphi_k(x)$$

可取:  $\varphi_k(x) = f(x) + \lambda_k p(x)$ ,

其中  $p(x)$  满足条件:

$$(1) \quad p(x) = 0 \quad x \in D;$$

$$(2) \quad p(x) > 0 \quad x \notin D。$$

$$\lambda_1 < \lambda_2 < \cdots < \lambda_k (\uparrow) \rightarrow +\infty$$





## 矩阵计算与优化 (最优化--有约束优化)

这样, 我们只需构造  $p(x)$ , 事实上,

因为  $g_j(x) \leq 0$

$$\Leftrightarrow \max\{g_j(x), 0\} = 0 \Leftrightarrow \max(\{g_j(x), 0\})^2 = 0$$

所以可设

$$p(x) = \sum_{j=1}^m (\max\{g_j(x), 0\})^2 = \sum_{j=1}^m \max^2\{g_j(x), 0\}$$

显然  $p(x)$  满足恰前面的条件(1)和(2)。

结论 1: 如果  $g_j(x) (j=1, 2, \dots, m)$  连续, 那么  $p(x)$  也连续。

事实上, 只须注意:

$$\max\{g_1(x), g_2(x)\} = \frac{g_1(x) + g_2(x) + |g_1(x) - g_2(x)|}{2}$$





## 矩阵计算与优化 (最优化--有约束优化)

结论 2: 如果 (1) 问题 (B):  $\min_{x \in R^n} \varphi_k(x)$  有最优解  $x^*(\lambda_k)$ ;

(2)  $x^*(\lambda_k) \in D$ , 即  $g_j(x^*(\lambda_k)) \leq 0$  ( $j=1,2,\dots,m$ )。

则  $x^*(\lambda_k)$  也是问题 (A):  $\min_{x \in D} f(x)$  的最优解。

证明: 因为  $x^*(\lambda_k)$  是 (B):  $\min_{x \in R^n} \varphi_k(x)$  的最优解。

所以  $\varphi_k(x^*(\lambda_k)) \leq \varphi_k(x), \quad \forall x \in R^n$ 。

又  $x^*(\lambda_k) \in D$ , 即  $g_j(x^*(\lambda_k)) \leq 0$  ( $j=1,2,\dots,m$ ),

所以  $p(x^*(\lambda_k)) = 0$ 。

$$\begin{aligned} \text{所以 } \forall x \in D, \text{ 有 } f(x^*(\lambda_k)) &= f(x^*(\lambda_k)) + \lambda_k p(x^*(\lambda_k)) \\ &= \varphi_k(x^*(\lambda_k)) \leq \varphi_k(x) \\ &= f(x) + \lambda_k p(x) = f(x) \end{aligned}$$

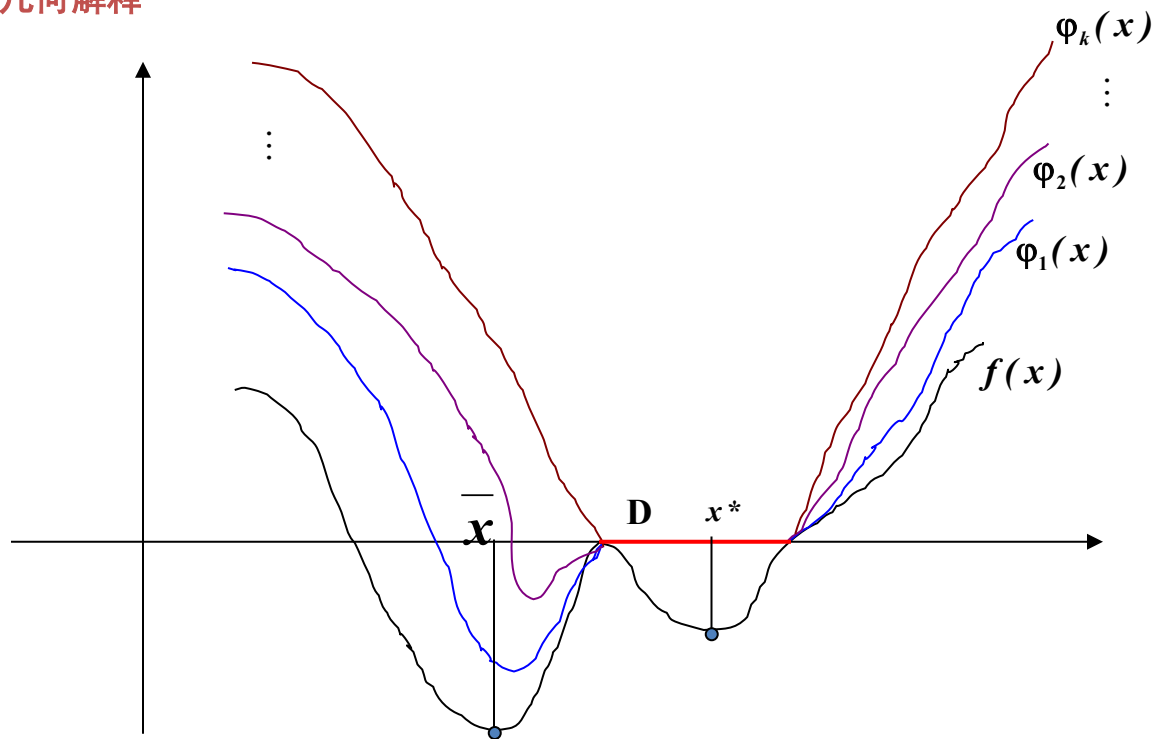
所以  $x^*(\lambda_k)$  是问题  $\min_{x \in D} f(x)$  的最优解。





# 矩阵计算与优化 (最优化--有约束优化)

## (1) 几何解释





## 矩阵计算与优化（最优化--有约束优化）

### （3）算法步骤（外点法）：

step1: 给定初始点  $x^0$ , 初始惩罚因子  $\lambda_1 > 0$  (可取  $\lambda_1 = 1$ ),  
精度  $\varepsilon > 0, k := 0$ 。

step2: 以  $x^k$  为初始点, 求解无约束优化问题

$$\min_{x \in R^n} \varphi_k(x) = f(x) + \lambda_k p(x) = f(x) + \lambda_k \sum_{j=1}^m \max^2(g_j(x), 0)$$

得到极小点为  $x^*(\lambda_k)$ , 记为  $x^{k+1}$ 。

step 3: 如果  $x^*(\lambda_k) \in D$ , 即  $\lambda_k p(x) \leq \varepsilon$ ,

则  $x^*(\lambda_k)$  就是问题 (A) :  $\min_{x \in D} f(x)$  的最优解, stop;

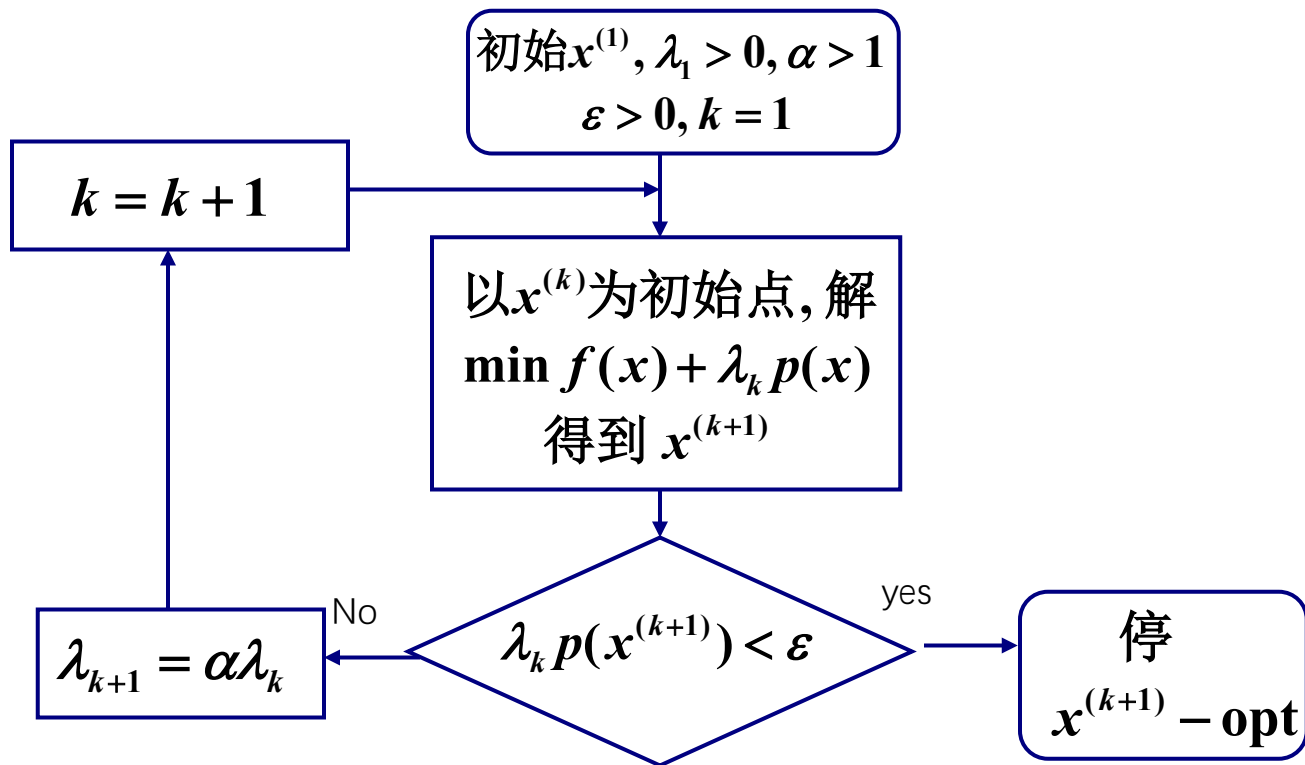
否则转 step 4.

step 4: 给定  $\lambda_{k+1} > \lambda_k$  (可取  $\lambda_{k+1} = \alpha \lambda_k$  这里  $\alpha > 1$  为惩罚因子的放大系数),  $k := k + 1$ , 转 step 2.





## 外点法框图





## 矩阵计算与优化 (最优化--有约束优化)

### (4) 应注意的问题

(a) 在step2中, 可用无约束优化问题的算法求解

$$\min_{x \in R^n} \varphi_k(x) = f(x) + \lambda_k p(x)$$

(b) 在实际计算中, 判断  $x^*(\lambda_k) \in D$  用  $|g_j(x^*(\lambda_k))| \geq \varepsilon$   
( $j = 1, 2, \dots, m$ ) 或  $\lambda_k p(x) < \varepsilon$ .

(c) 在算法中, 一开始  $\lambda_k$  不宜取的过大。否则会造成  
无约束优化问题  $\min_{x \in R^n} \varphi_k(x)$  求解困难 ( $\lambda_k$  越大,  
 $\varphi_k(x)$  的Hesse矩阵的条件数越坏)。

(d) 通常称:

$\varphi_k(x)$ : 增广函数,  $p(x)$ : 惩罚函数

$\lambda_k$ : 惩罚因子,  $\lambda_k p(x)$ : 惩罚项





## 矩阵计算与优化 (最优化--有约束优化)

例：试用外点法（外部惩罚函数法）求解如下优化问题

$$\min f(x) = (x-1)^2$$

$$\text{s.t. } x-2 \geq 0$$

解：构造增广函数  $\varphi_k(x)$  如下：

$$\varphi_k(x) = (x-1)^2 + \lambda_k \min(\{x-2, 0\})^2$$

$$= \begin{cases} (x-1)^2 & \text{if } x \geq 2 \\ (x-1)^2 + \lambda_k (x-2)^2 & \text{if } x < 2 \end{cases}$$

$$\frac{d\varphi_k(x)}{dx} = \begin{cases} 2(x-1) & \text{if } x \geq 2 \\ 2(x-1) + 2\lambda_k(x-2) & \text{if } x < 2 \end{cases}$$

$$\text{由 } \frac{d\varphi_k(x)}{dx} = 0 \text{ 可得：} 2(x-1) + 2\lambda_k(x-2) = 0$$





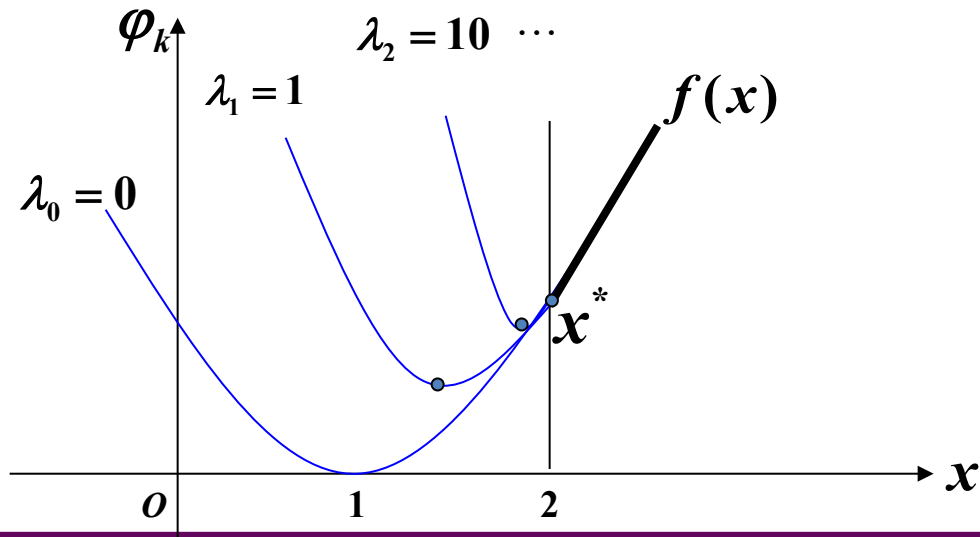
## 矩阵计算与优化 (最优化--有约束优化)

$$\text{所以 } x^k = x^*(\lambda_k) = \frac{1+2\lambda_k}{1+\lambda_k}$$

这就是对于固定的  $\lambda_k$ , 问题  $\min_{x \in R^n} \varphi_k(x)$  的最优解。

$$\lim_{k \rightarrow \infty} x^k = \lim_{k \rightarrow \infty} x^*(\lambda_k) = \lim_{k \rightarrow \infty} \frac{1+2\lambda_k}{1+\lambda_k} = 2 = x^*,$$

$x^*$  就是所求原问题的最优解。





### （7）一般模型的外点法

$$\begin{array}{ll} \min f(x) & \min f(x) \\ \text{s.t. } \begin{cases} g_i(x) \leq 0, i = 1, \dots, m \\ h_j(x) = 0, j = 1, \dots, p \end{cases} & \iff \text{s.t. } \begin{cases} g(x) \leq 0 \\ h(x) = 0 \end{cases} \end{array}$$

- 我们不难想到构造如下的惩罚函数

$$\begin{aligned} p(x) &= \sum_{j=1}^m (\max\{g_j(x), 0\})^2 + \sum_{k=1}^l (h_k(x))^2 \\ &= \sum_{j=1}^m \max^2\{g_j(x), 0\} + \sum_{k=1}^l h_k^2(x) \end{aligned}$$

- 算法步骤相同





### (8) 算法收敛性

结论 1. 若点列  $\{x^k\}$  是由外点法产生的, 则有

$$\varphi_{k+1}(x^{k+1}) \geq \varphi_k(x^k)$$

$$p(x^{k+1}) \leq p(x^k)$$

$$f(x^{k+1}) \geq f(x^k)$$

结论 2. 设  $f(x)$ 、 $g_j(x)$  ( $j=1, \dots, m$ ) 和  $h_k(x)$  ( $k=1, \dots, l$ ) 都是  $R^n$  上的连续函数, 则由外点法产生的点列  $\{x^k\}$  的任何极限点一定是所求问题的极小点。

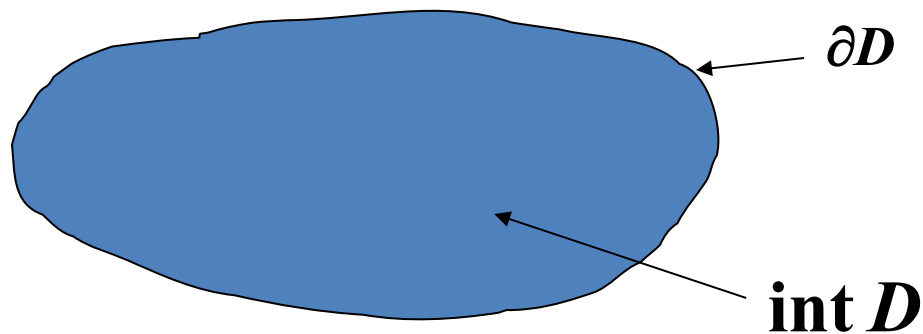




## 矩阵计算与优化 (最优化--有约束优化)

### 5、内点法 (障碍函数法)

#### (1) 集合结构



$$D = \partial D \cup \text{int } D = \text{边界点集} \cup \text{内点集}$$

$$\partial D = \{x \mid \text{至少存在一个 } j, \text{ 使得 } g_j(x) = 0\};$$

$$\text{int } D = \{x \mid g_j(x) < 0, \forall j\}.$$





### （2）算法思想

内点法（障碍函数法）的迭代点是在可行域点集内部移动的，对接近可行域边界上的点施加越来越大的惩罚，对可行域边界上的点施加无限大的惩罚，这好比边界是一道障碍物，阻碍迭代点穿越边界。

内点法要求可行点集的内点集合非空，否则算法无法运行。这样一来内点法只对不等式约束的优化问题才可能有效。







## 矩阵计算与优化 (最优化--有约束优化)

### (3) 算法分析

考虑如下优化问题:

$$\begin{aligned} \min & f(x) \\ \text{s.t. } & g_i(x) \leq 0, i = 1, \dots, m \end{aligned}$$

构造函数  $\psi_k(x) = f(x) + \mu_k q(x)$ ,

其中  $q(x)$  满足:

- (1)  $q(x) > 0$  if  $x \in \text{int } D$
- (2)  $q(x) \rightarrow +\infty$  if  $x \rightarrow \partial D$

转化为无约束优化问题:

$$\min_{x \in R^n} \psi_k(x) = f(x) + \mu_k q(x)$$

$$\mu_1 > \mu_2 > \dots > \mu_k (\downarrow) \rightarrow 0$$





$q(x)$ 一般取以下形式的函数（常取前两种）

$$q(x) = \sum_{j=1}^m -\frac{1}{g_j(x)} \quad \text{或} \quad q(x) = \sum_{j=1}^m \frac{1}{g_j^2(x)} \text{ 等};$$

$$q(x) = \sum_{j=1}^m \ln \frac{1}{-g_j(x)} \quad \text{或} \quad q(x) = -\sum_{j=1}^m \ln(-g_j(x)) \text{ 等}。$$

通常称：

$\psi_k(x)$ : 增广函数,       $q(x)$ : 障碍函数,

$\mu_k$ : 惩罚因子,       $\mu_k q(x)$ : 惩罚项。





## 矩阵计算与优化 (最优化--有约束优化)

(4) 算法步骤 (内点法) :

step1: 给定初始点  $x^0 \in \text{int } D$ , 初始惩罚因子  $\mu_1 > 0$ ,

(可取  $\mu_1 = 1$ ), 精度  $\varepsilon > 0, k := 0$ 。

step2: 以  $x^k$  为初始点, 求解无约束优化问题

$$\min_{x \in R^n} \psi_k(x) = f(x) + \mu_k q(x) = f(x) + \mu_k \sum_{j=1}^m \frac{1}{-g_j(x)}$$

得到其最优解  $x^*(\lambda_k)$ , 记为  $x^{k+1}$ 。

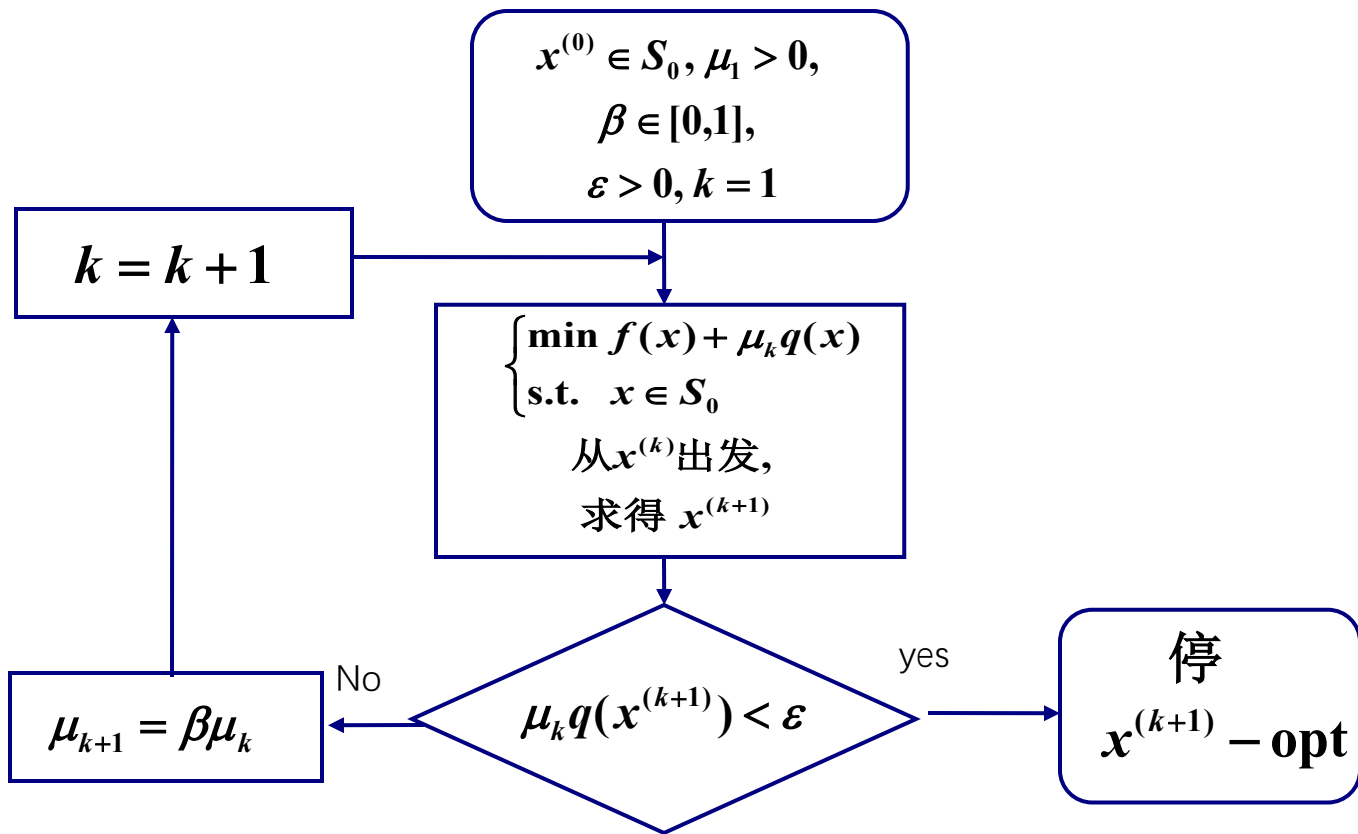
step 3: 如果  $\mu_k q(x^{k+1}) \leq \varepsilon$ , 则  $x^{k+1}$  就是问题

(A) :  $\min_{x \in D} f(x)$  的最优解, stop; 否则转 step 4。

step 4: 给定  $\mu_{k+1} < \mu_k$  (可取  $\mu_{k+1} = \beta \mu_k$  这里  $\beta < 1$  为惩罚因子的缩小系数),  $k := k + 1$ , 转 step 2。



## 内点法框图





## 矩阵计算与优化（最优化--有约束优化）

例 试用内点法（内部惩罚函数法）求解如下优化问题

$$\begin{aligned} \min f(x) &= \frac{1}{3}x^3 \\ \text{s.t. } x-1 &\geq 0 \end{aligned}$$

解 构造增广函数  $\psi_k(x)$  如下：

$$\psi_k(x) = x^3 + \mu_k \frac{1}{x-1}$$

由  $\frac{d\psi_k(x)}{dx} = x^2 - \frac{\mu_k}{(x-1)^2} = 0$  可得：

$$x(x-1) = \pm\sqrt{\mu_k}$$

因为  $x \geq 1$  所以  $x(x-1) = \sqrt{\mu_k}$ 。



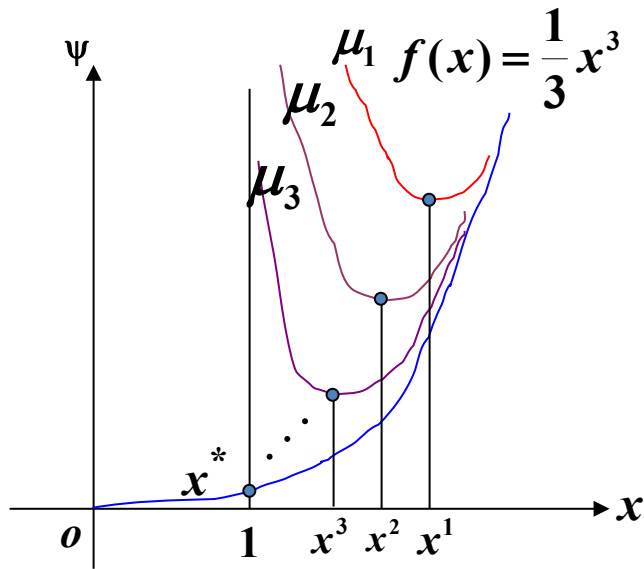


## 矩阵计算与优化（最优化--有约束优化）

因此  $x = \frac{1}{2}(1 \pm \sqrt{1 + 4\sqrt{\mu_k}})$

因为负根不满足条件，所以

$$x^{k+1} = \frac{1}{2}(1 + \sqrt{1 + 4\sqrt{\mu_k}})$$



由此可以得到：

$$x^* = \lim_{k \rightarrow \infty} x^{k+1} = \lim_{k \rightarrow \infty} \frac{1}{2}(1 + \sqrt{1 + 4\sqrt{\mu_k}}) = 1$$





## 矩阵计算与优化 (最优化--有约束优化)

### (5) 算法收敛性:

结论 1. 若点列  $\{x^k\}$  是由内点法产生的, 则有

$$\psi_{k+1}(x^{k+1}) \leq \psi_k(x^k)。$$

结论 2. 设  $f(x)$ 、 $g_j(x)(j=1, \dots, m)$  都是  $R^n$  上的连续函数, 则由内点法产生的点列  $\{x^k\}$  的任何极限点一定是所求问题的极小点。

### (6) 罚函数法的缺点

1. 当外点法 (或内点法) 的  $\lambda \rightarrow +\infty$  ( $\mu \rightarrow 0^+$ ) 时, 惩罚项  $\lambda_k p(x) \rightarrow +\infty \bullet 0$  (或  $\mu_k q(x) \rightarrow 0 \bullet +\infty$ ) 形式, 在计算上有困难;
2. 计算一系列无约束问题, 因而计算量大。





### (7)内、外点法的优缺点的比较

| 外点法                       | 内点法                                                      |
|---------------------------|----------------------------------------------------------|
| 1.任意 $x^{(0)} \in R^n$    | 1. $x^{(0)} \in S^0$                                     |
| 2.等式约束适用                  | 2. 等式约束不适用                                               |
| 3.惩罚项的二阶偏导在 $S$ 的边界上不存在   | 3. 障碍函数 $B(x)$ 在 $S^0$ 的可微阶数与 $g_i(x)$ 相同(可选用的无约束最优化方法广) |
| 4. 迭代中 $x^{(k)} \notin R$ | 4.迭代中 $x^{(k)} \in R$ (随时可取 $x^{(k)} \approx x^*$ )      |
| 5. 非凸规划适用                 | 5. 非凸规划适用                                                |





## 6. 乘子法

(1)等式约束

$$\left\{ \begin{array}{ll} \min & f(x) \quad f: R^n \rightarrow R \\ \text{s.t.} & h(x) = 0 \quad h: R^n \rightarrow R^l \\ x \in D & D \subseteq R^n \text{ 是一个集合, 常由简单约束构成。} \end{array} \right.$$

用Lagrange函数代替 $f(x)$ :  $L(x, v) = f(x) + \sum_{i=1}^l v_i h_i(x)$

乘子罚函数:

$$\phi(x, v, \mu) = f(x) + \sum_{i=1}^l v_i h_i(x) + \sum_{i=1}^l \mu_i h_i^2(x)$$

其中:  $v \in R^l$  为乘子,  $\mu \in R^l$  为罚因子。

乘子罚函数与Lagrange函数及惩罚函数的区别: 多一项。



乘子罚函数:

$$\phi(x, v, \mu) = f(x) + \sum_{i=1}^l v_i h_i(x) + \sum_{i=1}^l u_i h_i^2(x)$$

其中:  $v \in R^l$  为乘子,  $u \in R^l$  为罚因子。

$$\text{求解} \begin{cases} \min & \phi(x, v^{(k)}, u^{(k)}) \\ \text{s.t.} & x \in D \end{cases} \quad \text{得到 } x^{(k+1)}, k = 0, 1, 2, \dots$$

若  $h(x^{(k+1)}) = 0$  得到解  $x^{(k+1)}$  及乘子  $v^{(k)}$ ;

否则 调整  $v^{(k)}$  及  $\mu^{(k)}$ 。





可以证明:

存在 $\mu'$ , 当 $\mu > \mu'$ 时 存在 $v^*$

$$\begin{cases} \min & \phi(x, v^*, \mu) \\ \text{s.t.} & x \in D \end{cases}$$

的最优解, 即原问题的解。

(2)等式、不等式约束

$$\begin{cases} \min & f(x) \\ \text{s.t.} & g(x) \leq 0 \\ & h(x) = 0 \\ & x \in D \end{cases} \xrightarrow{\text{引入松弛变量}} \begin{cases} \min & f(x) \\ \text{s.t.} & g(x) + z = 0 \\ & h(x) = 0 \\ & x \in D' = \left\{ \begin{pmatrix} x \\ z \end{pmatrix} \mid x \in D, z \geq 0 \right\} \end{cases}$$





## 矩阵计算与优化 (最优化--有约束优化)

算法步骤 (乘子罚函数法) :

step1: 给定初始点  $x^0$ , 初始乘子向量  $v^{(1)}$ , 初始罚因子  $u^{(1)}$  及其放大系数  $\alpha > 1$  (可取  $2 \leq \alpha \leq 10$ ), 常数  $r \in (0, 1)$ , 精度  $\varepsilon > 0$ , 令  $k := 1$ .

step2: 以  $x^{k-1}$  为初始点, 求解无约束优化问题

$$\min_{x \in R^n} \phi(x, v^{(k)}, u^{(k)}) = f(x) + \sum_{i=1}^l v_i^{(k)} h_i(x) + u^{(k)} \sum_{i=1}^l h_i^2(x)$$

得到其最优解  $x^*(u^{(k)}, v^{(k)})$ , 记为  $x^k$ .

step 3: 如果  $\|h(x^k)\| \leq \varepsilon$ , 则  $x^k$  就是最优解, stop; 否则, 计算

$\beta = \|h(x^k)\| / \|h(x^{k-1})\|$ , 若  $\beta \leq r$ , 转 step 4,

若  $\beta > r$ , 令  $u^{(k+1)} = \alpha u^{(k)}$ , 转 step 4.

step 4: 计算  $v_j^{(k+1)} = v_j^{(k)} + u^{(k)} h_j(x)$ ,  $j = 1, 2, \dots, m$ ,  $k := k + 1$ , 转 step 2.





## 矩阵计算与优化 (最优化--有约束优化)

例：问题  $\min \frac{1}{2}x_1^2 + \frac{1}{6}x_2^2$  的最优解为  $x^* = (0.25, 0.75)$ , 分别用惩罚函数法和乘子法求它的迭代点列。

s.t.  $x_1 + x_2 = 1$

解：1. 惩罚函数法。对于惩罚函数

$$\psi(x) = \frac{1}{2}x_1^2 + \frac{1}{6}x_2^2 + u_k(x_1 + x_2 - 1)^2$$

可求得最优解为：  $x^{k*} = \left( \frac{2u_k}{1+8u_k}, \frac{6u_k}{1+8u_k} \right)$

2. 乘子法。对于乘子罚函数

$$\phi(x) = \frac{1}{2}x_1^2 + \frac{1}{6}x_2^2 + v_k(x_1 + x_2 - 1) + u_k(x_1 + x_2 - 1)^2$$

可求得最优解为：  $x^{k*} = \left( \frac{2u_k - v_k}{1+8u_k}, \frac{3(2u_k - v_k)}{1+8u_k} \right)$

取  $u_k = 0.05 \times 2^k$ ,  $v_0 = 0$ ,  $v_{k+1} = v_k + u_k(x_1^{(k)} + x_2^{(k)} - 1)$ ,  $k \geq 1$ , 得下表





| $k$ | $x^k$ (惩罚函数法)    | $x^k$ (乘子法)      |
|-----|------------------|------------------|
| 0   | (0.0714, 0.2142) | (0.0714, 0.2142) |
| 1   | (0.1111, 0.3333) | (0.1507, 0.4523) |
| 2   | (0.1538, 0.4615) | (0.2118, 0.6355) |
| 3   | (0.1904, 0.5714) | (0.2409, 0.7227) |
| 4   | (0.2162, 0.6486) | (0.2487, 0.7463) |
| 5   | (0.2318, 0.6956) | (0.2499, 0.7497) |
| 6   | (0.2406, 0.7218) | (0.2499, 0.7499) |
| 7   | (0.2452, 0.7356) |                  |
| 8   | (0.2475, 0.7427) |                  |
| 9   | (0.2487, 0.7463) |                  |
| 10  | (0.2492, 0.7481) |                  |
| 11  | (0.2496, 0.7490) |                  |
| 12  | (0.2498, 0.7495) |                  |
| 13  | (0.2499, 0.7497) |                  |
| 14  | (0.2499, 0.7498) |                  |
| 15  | (0.2499, 0.7499) |                  |

从表中可见,  $x^{k*}$  比  $x^k$  近于  $x^*$  的速度慢得多, 用乘子法迭代6次就达到惩罚函数法迭代15次的效.

这里, 惩罚因子在惩罚函数法中要增大到  $u_{15} = 3276.8$ , 而在乘子法中只要增大到  $u_6 = 6.4$ .

相比之下, 乘子法不需过分地增大惩罚因子, 确实比惩罚函数法有效很多.





### Matlab求解约束非线性规划

非线性有约束的多元函数的标准形式为：

$$\begin{array}{ll} \min_x & f(x) \\ \text{sub.to} & \left. \begin{array}{l} C(x) \leq 0 \\ Ceq(x) = 0 \end{array} \right\} \text{非线性约束} \\ & \left. \begin{array}{l} Ax \leq b \\ Aeq \cdot x = 0 \end{array} \right\} \text{线性约束} \\ & lb \leq x \leq ub \end{array}$$

其中：

$x$ 、 $b$ 、 $beq$ 、 $lb$ 、 $ub$ 是向量， $A$ 、 $Aeq$ 为矩阵， $C(x)$ 、 $Ceq(x)$ 是约束向量的函数， $f(x)$ 为目标函数， $f(x)$ 、 $C(x)$ 、 $Ceq(x)$ 可以是非线性函数。





## 矩阵计算与优化 (最优化--有约束优化)

函数 **fmincon**

格式  $x = \text{fmincon}(\text{fun}, x_0, A, b)$

$x = \text{fmincon}(\text{fun}, x_0, A, b, A_{eq}, b_{eq})$

$x = \text{fmincon}(\text{fun}, x_0, A, b, A_{eq}, b_{eq}, lb, ub)$

$x = \text{fmincon}(\text{fun}, x_0, A, b, A_{eq}, b_{eq}, lb, ub, \text{nonlcon})$

$x = \text{fmincon}(\text{fun}, x_0, A, b, A_{eq}, b_{eq}, lb, ub, \text{nonlcon}, \text{options})$

$[x, fval] = \text{fmincon}(\dots)$

$[x, fval, \text{exitflag}] = \text{fmincon}(\dots)$

$[x, fval, \text{exitflag}, \text{output}] = \text{fmincon}(\dots)$

$[x, fval, \text{exitflag}, \text{output}, \lambda] = \text{fmincon}(\dots)$

$[x, fval, \text{exitflag}, \text{output}, \lambda, \text{grad}] = \text{fmincon}(\dots)$

$[x, fval, \text{exitflag}, \text{output}, \lambda, \text{grad}, \text{hessian}] = \text{fmincon}(\dots)$







## 矩阵计算与优化 (最优化--有约束优化)

例1  $\min f = -x_1 - 2x_2 + \frac{1}{2}x_1^2 + \frac{1}{2}x_2^2$

$$2x_1 + 3x_2 \leq 6$$

$$\text{s.t. } x_1 + 4x_2 \leq 5$$

$$x_1, x_2 \geq 0$$

解: (1)写成标准形式:

$$\min f = -x_1 - 2x_2 + \frac{1}{2}x_1^2 + \frac{1}{2}x_2^2$$

$$\text{s.t. } \begin{pmatrix} 2x_1 + 3x_2 - 6 \\ x_1 + 4x_2 - 5 \end{pmatrix} \leq \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix} \leq \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$





## 矩阵计算与优化 (最优化--有约束优化)

(2)先建立M-文件 fun1.m:

```
function f=fun1(x);  
f=-x(1)-2*x(2)+(1/2)*x(1)^2+(1/2)*x(2)^2
```

(3)再建立主程序youh1.m:

```
x0=[1;1];  
A=[2 3 ;1 4]; b=[6;5];  
Aeq=[];beq=[];  
LB=[0;0]; UB=[];  
[x,fval]=fmincon('fun1',x0,A,b,Aeq,beq,LB,UB)
```

(4)在命令窗口中输入youh1,得运算结果为:

```
x = 0.7647    1.0588  
fval = -2.0294
```





## 矩阵计算与优化 (最优化--有约束优化)

例 2: 求下面问题在初始点  $(0, 1)$  处的最优解

$$\min \quad x_1^2 + x_2^2 - x_1 x_2 - 2x_1 - 5x_2$$

$$\text{s.t.} \quad -(x_1 - 1)^2 + x_2 \geq 0$$

$$2x_1 - 3x_2 + 6 \geq 0$$

解: 约束条件的标准形式为  $(x_1 - 1)^2 - x_2 \leq 0$   
 $-2x_1 + 3x_2 \leq 6$

(1) 在MATLAB编辑器中建立非线性约束函数文件:

```
function [c, ceq]=nlcon(x)
```

```
c=(x(1)-1)^2-x(2);
```

```
ceq=[]; %无等式约束
```





## 矩阵计算与优化 (最优化--有约束优化)

(1) 在MATLAB编辑器中建立非线性约束函数文件:

```
function [c, ceq]=nlcon (x)
c=(x(1)-1)^2-x(2);
ceq=[];    %无等式约束
```

(2) 在命令窗口键入如下命令或建立M文件:

```
fun2='x(1)^2+x(2)^2-x(1)*x(2)-2*x(1)-5*x(2)';    %目标函数
x0=[0 1];
A=[-2 3];    %线性不等式约束
b=6;
Aeq=[];    %无线性等式约束
beq=[];
lb=[];    % x没有下、上界
ub=[];
[x,fval,exitflag,output,lambda,grad,hessian]
=fmincon(fun2,x0,A,b,Aeq,beq,lb,ub,@nlcon)
```





## 矩阵计算与优化（最优化--有约束优化）

```
则结果为
x =
    3    4
fval =
   -13
exitflag =    %解收敛
    1
output =
    iterations: 2
    funcCount: 9
    stepsize: 1
    algorithm: 'medium-scale: SQP, Quasi-Newton, line-search'
    firstorderopt: []
    cgiterations: []
lambda =
    lower: [2x1 double] %x下界有效情况，通过lambda.lower可查看。
    upper: [2x1 double] %x上界有效情况，为0表示约束无效。
    eqlin: [0x1 double] %线性等式约束有效情况，不为0表示约束有效。
    eqnonlin: [0x1 double] %非线性等式约束有效情况。
    ineqlin: 2.5081e-008 %线性不等式约束有效情况。
    ineqnonlin: 6.1938e-008 %非线性不等式约束有效情况。
grad =    %目标函数在最小值点的梯度
    1.0e-006 *
   -0.1776
    0
hessian =    %目标函数在最小值点的Hessian值
    1.0000   -0.0000
   -0.0000    1.0000
```





## 矩阵计算与优化（最优化--有约束优化）

二次规划问题（quadratic programming）的Matlab解

二次规划问题的标准形式为：

$$\begin{aligned} \min \quad & \frac{1}{2} x' H x + f' x \\ \text{sub.to} \quad & A \cdot x \leq b \\ & Aeq \cdot x = beq \\ & lb \leq x \leq ub \end{aligned}$$

其中，H、A、Aeq为矩阵，f、b、beq、lb、ub、x为向量。

其它形式的二次规划问题都可转化为标准形式。





## 矩阵计算与优化（最优化--有约束优化）

### 函数 quadprog

格式:  $x = \text{quadprog}(H,f,A,b)$  %其中 $H,f,A,b$ 为标准形中的参数,  
 $x$ 为目标函数的最小值。

$x = \text{quadprog}(H,f,A,b,Aeq,beq)$  % $Aeq,beq$ 满足等约束条件

$x = \text{quadprog}(H,f,A,b,Aeq,beq,lb,ub)$  %  $lb,ub$ 分别为为 $x$ 的下上界。

$x = \text{quadprog}(H,f,A,b,Aeq,beq,lb,ub,x0)$  % $x0$ 为设置的初值

$x = \text{quadprog}(H,f,A,b,Aeq,beq,lb,ub,x0,options)$  %  $options$ 为指定  
的优化参数

$[x,fval] = \text{quadprog}(\dots)$  % $fval$ 为目标函数最优值

$[x,fval,exitflag] = \text{quadprog}(\dots)$  %  $exitflag$ 与线性规划中参数意义  
相同

$[x,fval,exitflag,output] = \text{quadprog}(\dots)$  %  $output$ 与线性规划中参  
数意义相同

$[x,fval,exitflag,output,lambda] = \text{quadprog}(\dots)$  %  $lambda$ 与线性  
规划中参数意义相同





## 矩阵计算与优化 (最优化--有约束优化)

例： 求解下面二次规划问题

$$\min \quad f(x) = \frac{1}{2}x_1^2 + x_2^2 - x_1x_2 - 2x_1 - 6x_2$$

sub.to

$$x_1 + x_2 \leq 2$$

$$-x_1 + 2x_2 \leq 2$$

$$2x_1 + x_2 \leq 3$$

$$0 \leq x_1, \quad 0 \leq x_2$$

解：  $f(x) = \frac{1}{2}x' H x + f' x$ , 则  $H = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}$ ,  $f = \begin{bmatrix} -2 \\ -6 \end{bmatrix}$ ,  $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$

在MATLAB中实现如下：

```
H = [1 -1; -1 2];
```

```
f = [-2; -6];
```

```
A = [1 1; -1 2; 2 1];
```

```
b = [2; 2; 3];
```

```
lb = zeros(2,1);
```

```
[x,fval,exitflag,output,lambda] = quadprog(H,f,A,b,[],[],lb)
```







## 矩阵计算与优化 (最优化--有约束优化)

结果为:

x = %最优解

0.6667

1.3333

fval = %最优值

-8.2222

exitflag = %收敛

1

output =

iterations: 3

algorithm:

'medium-scale:

active-set'

firstorderopt: [ ]

cgiterations: [ ]

lambda =

lower:[2x1 double]

upper:[2x1 double]

eqlin: [0x1 double]

ineqlin: [3x1 double]

>> lambda.ineqlin

ans =

3.1111

0.4444

0

>> lambda.lower

ans =

0

0

说明: 第 1、2 个约束条件  
有效, 其余无效。





### 非线性不等式约束

所用函数：

$$\min_x f(x)$$

$$c(x) \leq 0, ceq(x) = 0$$

$$A \cdot x \leq b, Aeq \cdot x = beq, lb \leq x \leq ub$$

式中：

$x, b, beq, lb, ub$  —— 向量；

$A, Aeq$  —— 矩阵；

$c(x), ceq(x)$  —— 返回向量的函数；

$f(x)$  —— 返回标量的函数；

$f(x), c(x), ceq(x)$  —— 可以是非线性函数。





## 矩阵计算与优化（最优化--有约束优化）

$$\mathbf{x} = \text{fmincon}(\text{fun}, \mathbf{x0}, \mathbf{A}, \mathbf{b})$$

$\mathbf{x0}$ 为起始点，寻找函数 $\text{fun}$ 的最小值，约束条件为线性不等式约束 $\mathbf{A}^* \mathbf{x} \leq \mathbf{b}$ 。  
 $\mathbf{x0}$ 可以是标量、向量或矩阵。

$$\mathbf{x} = \text{fmincon}(\text{fun}, \mathbf{x0}, \mathbf{A}, \mathbf{b}, \mathbf{Aeq}, \mathbf{beq})$$

寻找函数 $\text{fun}$ 的最小值，约束条件为线性等式约束 $\mathbf{Aeq}^* \mathbf{x} = \mathbf{beq}$ 以及线性不等式约束 $\mathbf{A}^* \mathbf{x} \leq \mathbf{b}$ 。如果无线性不等式约束设置 $\mathbf{A}=[]$ 、 $\mathbf{b}=[]$ 。

$$\mathbf{x} = \text{fmincon}(\text{fun}, \mathbf{x0}, \mathbf{A}, \mathbf{b}, \mathbf{Aeq}, \mathbf{beq}, \mathbf{lb}, \mathbf{ub})$$

定义了变量 $\mathbf{x}$ 的上下界，使最优解 $\mathbf{x}$ 为 $\mathbf{lb} \leq \mathbf{x} \leq \mathbf{ub}$ 。如果等式约束不存在，设置 $\mathbf{Aeq}=[]$ 、 $\mathbf{beq}=[]$  如果 $\mathbf{x}(i)$ 无下界，设置 $\mathbf{lb}(i)=-\text{Inf}$ ；如果 $\mathbf{x}(i)$ 无上界，设置 $\mathbf{ub}(i)=\text{Inf}$ 。





## 矩阵计算与优化（最优化--有约束优化）

**$\mathbf{x} = \text{fmincon}(\text{fun}, \mathbf{x0}, \mathbf{A}, \mathbf{b}, \mathbf{Aeq}, \mathbf{beq}, \mathbf{lb}, \mathbf{ub}, \text{nonlcon})$**

**nonlcon**中定义了非线性不等式 $\mathbf{c}(\mathbf{x})$ 或等式 $\mathbf{ceq}(\mathbf{x})$ 约束。优化结果使 $\mathbf{c}(\mathbf{x}) \leq 0$ 、 $\mathbf{ceq}(\mathbf{x}) = 0$ 。  
如果变量 $\mathbf{x}$ 无上下界，设置 $\mathbf{lb}=[]$ 、 $\mathbf{ub}=[]$ 。

**$\mathbf{x} = \text{fmincon}(\text{fun}, \mathbf{x0}, \mathbf{A}, \mathbf{b}, \mathbf{Aeq}, \mathbf{beq}, \mathbf{lb}, \mathbf{ub}, \text{nonlcon}, \text{options})$**

优化选项在结构**options**中定义。使用**optimset** 函数定义**options**. 如果没有非线性不等式和等式约束，设置**nonlcon** = []。

**$[\mathbf{x}, \mathbf{fval}] = \text{fmincon}(\dots)$**

返回最优解（ $\mathbf{x}$ ， $\mathbf{fval}$ ）。

**$[\mathbf{x}, \mathbf{fval}, \text{exitflag}] = \text{fmincon}(\dots)$**

返回描述**fmincon** 结束的原因的标识符**exitflag**。





## 矩阵计算与优化（最优化--有约束优化）

**$[x, fval, exitflag, output] = fmincon(...)$**

结构`output`中包含优化信息。

**$[x, fval, exitflag, output, lambda] = fmincon(...)$**

返回最优解`x`的拉格朗日乘子，存储在结构`lambda`中。

**$[x, fval, exitflag, output, lambda, grad] = fmincon(...)$**

在`grad`中包含函数`fun`在`x`处的梯度值。

**$[x, fval, exitflag, output, lambda, grad, hessian] = fmincon(...)$**

在`hessian`中包含函数`fun`在`x`处的Hessian矩阵值。





## 矩阵计算与优化 (最优化--有约束优化)

| exitflag | 算法终止原因                                                                      |
|----------|-----------------------------------------------------------------------------|
| 1        | First order optimality conditions were satisfied to the specified tolerance |
| 2        | $x$ 的变化小于指定值                                                                |
| 3        | 目标函数的变化小于指定值                                                                |
| 4        | 搜索方向的绝对值小于特定的容许值并且约束偏差小于 options.TolCon                                     |
| 5        | 方向导数的绝对值小于特定的容许值并且约束偏差小于 options.TolCon                                     |
| 0        | 迭代数超过options.MaxIter或函数的计算数超过 options.FunEvals                              |
| -1       | 算法被输出函数终止                                                                   |
| -2       | 找不到可行点                                                                      |





## 矩阵计算与优化 (最优化--有约束优化)

|                             |                   |
|-----------------------------|-------------------|
| <b>lambda</b>               | 在最优点 $x$ 处的拉格朗日乘子 |
| <b>lower</b>                | 下界 <b>lb</b>      |
| <b>upper</b>                | 上界 <b>ub</b>      |
| <b>ineqlin</b>              | 线性不等式             |
| <b>eqlin</b>                | 线性等式              |
| <b>ineqnonlin</b>           | 非线性不等式            |
| <b>Nonlinear equalities</b> | 非线性等式             |





## 矩阵计算与优化 (最优化--有约束优化)

|                      |                                    |
|----------------------|------------------------------------|
| <b>output</b>        | 包含优化信息的结构                          |
| <b>iterations</b>    | 迭代数                                |
| <b>funcCount</b>     | 函数计算数                              |
| <b>lgorithm</b>      | 使用的算法                              |
| <b>cgiterations</b>  | PCG迭代数(large-scale algorithm only) |
| <b>stepsize</b>      | 最终步长(medium-scale algorithm only)  |
| <b>firstorderopt</b> | 一阶最优性的测量                           |







### 问题

$$\underset{\mathbf{x}}{\text{minimize}} \ f(\mathbf{x}) = e^{x_1} (4x_1^2 + 2x_2^2 + 4x_1x_2 + 2x_2 + 1)$$

约束条件:

$$x_1x_2 - x_1 - x_2 \leq -1.5$$

$$x_1x_2 \geq -10$$





### Matlab工具箱求解过程

不能在命令行把非线性的约束传给fmincon。但可创建一个M-file, confun.m, 这个文件返回当前x的两个约束条件的变量c。接着调用带约束的优化函数fmincon。由于fmincon要求约束条件被写成  $c(x) \leq 0$  的形式, 因此必须重写以下形式的约束。

$$\begin{aligned} x_1 x_2 - x_1 - x_2 + 1.5 &\leq 0 \\ -x_1 x_2 - 10 &\leq 0 \end{aligned}$$





步骤1：写一个约束条件的confun.m 文件。

```
function [c, ceq] = confun(x)
```

```
c = [1.5 + x(1)*x(2) - x(1) - x(2); -x(1)*x(2) - 10];
```

```
%非线性不等式约束
```

```
ceq = [];
```

```
% 非线性等式约
```

束





## 矩阵计算与优化 (最优化--有约束优化)

步骤2: 写一个目标函数的M文件objfun.m

```
function f = objfun(x)
```

```
f = exp(x(1))*(4*x(1)^2+2*x(2)^2+4*x(1)*x(2)+2*x(2)+1);
```

步骤3: 调用带约束的优化程序。

```
x0 = [-1,1];
```

% 起始点

```
options = optimset('LargeScale','off','Display','iter');
```

```
[x,fval,exitflag,output]=
```

```
fmincon(@objfun,x0,[],[],[],[],[],[],@confun,options)
```





## 矩阵计算与优化（最优化--有约束优化）

### 运行结果

**x =**

**-9.5474    1.0474**

**fval =**

**0.0236**

**exitflag =**

**1**

**output =**

**iterations: 8**

**funcCount: 36**

**stepsize: 1**

**algorithm: 'medium-scale: SQP, Quasi-Newton, line-search'**

**firstorderopt: 8.5125e-007**

**cgiterations: []**

**message: [1x144 char]**





## 矩阵计算与优化 (最优化--有约束优化)

### 运行结果分析

36 次函数调用后, 最优解为

$\mathbf{x} =$

**-9.5474 1.0474**

$\mathbf{fval} =$

**0.0236**

也可计算最优解处的约束。

$[\mathbf{c}, \mathbf{ceq}] = \mathbf{confun}(\mathbf{x})$

返回  $\mathbf{c} =$

**$1.0\text{e-}14 *$**

**0.1110**

**-0.1776**

$\mathbf{ceq} =$

**[]**

可注意到约束值小于等于0, 即 $\mathbf{x}$ 满足。





### 带边界的约束

问题

$$\underset{\mathbf{x}}{\text{minimize}} f(\mathbf{x}) = e^{x_1} (4x_1^2 + 2x_2^2 + 4x_1x_2 + 2x_2 + 1)$$

约束条件:

$$x_1 \geq 0$$

$$x_2 \geq 0$$





### Matlab工具箱求解过程

通过对约束优化函数定义简单的边界约束，变量 $x$ 可以被限制在某一范围。对于函数fmincon，命令

$x = \text{fmincon}(@\text{objfun}, x0, [], [], [], [], \text{lb}, \text{ub}, @\text{confun}, \text{options});$

限制  $x$  为  $\text{lb} \leq x \leq \text{ub}$ 。







## 矩阵计算与优化 (最优化--有约束优化)

步骤1: 写一个约束条件的`confun.m` 文件。

```
function [c, ceq] = confun(x)
```

```
c = [1.5 + x(1)*x(2) - x(1) - x(2); -x(1)*x(2) - 10];
```

%非线性不等式约束

```
ceq = [];
```

% 非线性等式约束

步骤2: 写一个目标函数的M文件`objfun.m`

```
function f = objfun(x)
```

```
f = exp(x(1))*(4*x(1)^2+2*x(2)^2+4*x(1)*x(2)+2*x(2)+1);
```





## 矩阵计算与优化（最优化--有约束优化）

步骤3：调用带约束的优化程序

为使方程中 $x$ 大于0，使用如下命令。

```
x0 = [-1,1]; % 估计问题的起始点
lb = [0,0]; % 设置下界
ub = [ ]; % 无上界
options = optimset('LargeScale','off','Display','iter');
[x,fval,exitflag,output]=
fmincon(@objfun,x0,[],[],[],[],lb,ub,@confun,options)
[c, ceq] = confun(x)
```

注意：为了向fmincon传递下界作为第七个变量，必须定义第三个到第六个变量的值。本例中因为没有线性不等式或线性等式约束，定义这些变量为空。





## 矩阵计算与优化 (最优化--有约束优化)

```
x =  
    0    1.5000  
fval =  
    8.5000  
exitflag =  
    1  
output =  
    iterations: 3  
    funcCount: 15  
    stepsize: 1  
    algorithm: 'medium-scale: SQP, Quasi-Newton, line-search'  
    firstorderopt: 1.6307e-012  
    cgiterations: []  
    message: [1x144 char]  
c =  
    0  
   -10  
ceq =  
    []
```





## 矩阵计算与优化 (最优化--有约束优化)

### 运行结果分析

当  $lb$  或  $ub$  比  $x$  包含的元素少, 仅仅  $x$  中对应的元素被约束。如果仅仅变量的一部分被约束, 在  $lb$  中使用  $-\text{inf}$  作为无约束的下边界向量, 在  $ub$  中使用  $\text{inf}$  作为无约束的上变量。例如:

$lb = [-\text{inf } 0];$

$ub = [10 \text{ inf}];$

边界  $x_1 \leq 10$ 、 $x_2 \geq 0$ ,  $x_1$  没有下界和  $x_2$  没有上界。使用  $\text{inf}$  和  $-\text{inf}$  比使用很大的正数或很大的负数作为缺少的边界, 得到更好的数值结果。

注意: 进一步限制了搜索空间后, 寻找最终解的函数计算次数减少 (66  $\rightarrow$  36  $\rightarrow$  15)。当问题有较多的约束和边界限制时, 通常有较少的函数计算。因为最优化关于步长和可行性区域, 约束问题比无约束问题有更好的判断。因此, 尽可能约束和限制问题较好, 这可促进快速收敛。





## 矩阵计算与优化 (最优化--有约束优化)

为了显示每次迭代的输出，键入

```
options = optimset('Display', 'iter');
```

此命令设置Display option 的值为 ‘iter’。

这个命令的设置使工具箱每次迭代都显示结果。也可以关闭任何输出结果(display 设为 ‘off’), 仅仅在最后显示输出结果 (display 设为 ‘final’), 或在问题不收敛时显示 (display 设为 'notify')。

当选项Display设为'iter', 运行结果呈以下形式。

|      |         |      | max        |           | Directional | First-order |           |
|------|---------|------|------------|-----------|-------------|-------------|-----------|
| Iter | F-count | f(x) | constraint | Step-size | derivative  | optimality  | Procedure |





### 等式约束

非线性等式约束必须与非线性不等式约束同时在M文件中计算。

### 问题

$$\underset{x}{\text{minimize}} \ f(x) = e^{x_1} (4x_1^2 + 2x_2^2 + 4x_1x_2 + 2x_2 + 1)$$

$$x_1^2 + x_2 = 1$$

$$x_1x_2 \geq -10$$





### Matlab工具箱求解过程

约束条件改写为：

$$\begin{aligned}x_1^2 + x_2 - 1 &= 0 \\ -x_1 x_2 - 10 &\leq 0\end{aligned}$$

步骤1：写M文件objfun.m

```
function f = objfun(x)
```

```
f = exp(x(1))*(4*x(1)^2+2*x(2)^2+4*x(1)*x(2)+2*x(2)+1);
```





步骤2：写非线性约束的 M文件constraints

```
function [c, ceq] = confuneq(x)
```

```
c = -x(1)*x(2) - 10;           %非线性不等式约束
```

```
ceq = x(1)^2 + x(2) - 1;       %非线性等式约束
```







步骤3：调用约束优化程序

```
x0 = [-1,1];                                % 估计初始点

options = optimset('LargeScale','off', 'Display','iter');

[x,fval] = fmincon(@objfun,x0,[],[],[],[],[],[], @confuneq,options)

[c,ceq] = confuneq(x)                        % 检查x点的约束值
```





## 矩阵计算与优化（最优化--有约束优化）

**x =**

**-0.7529    0.4332**

**fval =**

**1.5093**

**c =**

**-9.6739**

**ceq =**

**6.3038e-009**

运行结果分析

在约束的缺省误差**1.0e-006**范围内为0，**c**小于等于0。





感谢聆听!